

IUT de Paris Descartes – Base de la Programmation Objet

Travaux Pratiques – Sujet n°11

Objectifs

- Revoir le "méta-principe" et la gestion de dépendances.
- Déplacer les instanciations au bon endroit : fabrique ou injection de dépendance.

Les chenilles suite et fin

Nous continuons à améliorer l'application permettant l'animation des déplacements de chenille. L'architecture de l'application est à présent celle de la figure 1.

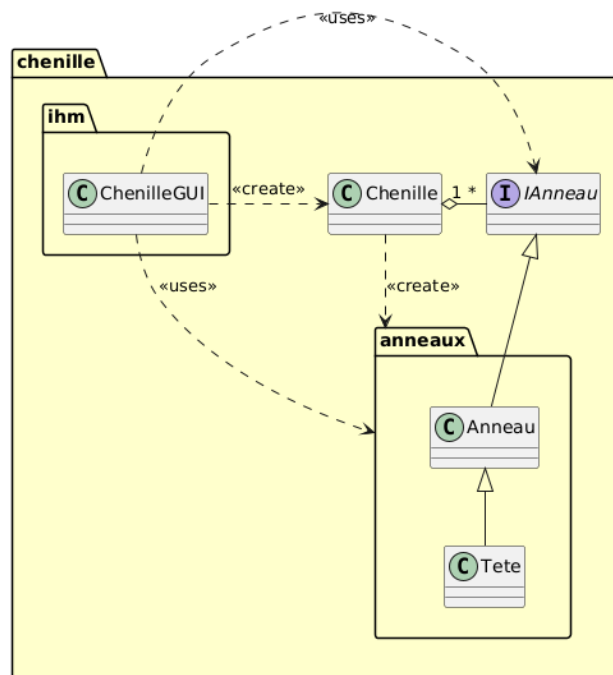


FIGURE 1 – Architecture à améliorer

Rappel : Les principes SOLID qui ont été appliqués pour obtenir cette architecture seront vus en 2e année mais peuvent être résumés par cet unique méta-principe : **cacher ce qui est variable et les cas particuliers du reste de l'application et notamment de ce qui est plus stable.**

Ce qui change c'est l'ensemble des classes d'anneaux. Elles sont donc regroupées dans le paquetage chenille.anneaux qui le **paquetage instable**. Le paquetage chenille est le **paquetage stable** (qui change

moins vite). Le paquetage chenille.ihm est aussi un paquetage instable car on veut pouvoir changer l'ihm fréquemment même si ce n'est pas le sujet principal de ce TP.

Idéalement aucun paquetage ne doit dépendre d'un paquetage instable (sauf s'ils varient au même rythme) mais il est difficile d'empêcher l'ihm de dépendre de ce qui est affiché. En revanche **ce qui est affiché ne doit PAS dépendre de l'ihm**. C'est déjà le cas et cela doit le rester.

Lors de la séance précédente, nous avons presque réussi à supprimer les dépendances du paquetage chenille vers chenille.anneaux. Il ne reste plus qu'à déplacer les créations d'instances d'anneaux de la classe Chenille vers une classe appropriée dans un paquetage approprié.

1. Créez un projet et importez les fichiers sources à partir du lien indiqué sur Moodle. Faites tourner ChenilleGui.main pour vérifier que l'application fonctionne correctement.
2. On veut supprimer les dépendances de Chenille vers le paquetage chenille.anneaux. Dans un premier temps, commentez les imports dans Chenille.java pour mettre en exergue les dépendances interdites c'est à dire les **new** sur des classes d'anneaux. Créez une méthode statique **creerAnneau** en lui passant (entre autres) la position que va occuper l'anneau dans la chenille. Si cette position est 0 créez une tête sinon un anneau normal. Cette méthode sera définie dans une classe appropriée qui deviendra une **fabrique** d'anneaux.
3. Grâce à cette fabrique qui se charge des créations d'anneaux, la classe Chenille n'a plus de dépendance sur les classes d'anneaux et ne devrait pas avoir à être modifiée quand nous ajouterons une nouvelle classe d'anneaux. Mais le problème semble avoir seulement été déplacé et non pas réglé. Pourquoi est-ce mieux malgré tout ?
4. Une fabrique peut être intéressante pour centraliser les connaissances concernant quel type d'anneaux créer selon les circonstances, ici en fonction de leur position dans la chenille. Néanmoins il reste à présent une dépendance de Chenille vers le paquetage IHM qui est instable (ou vers un paquetage qui dépend de tout façon de chenille.anneaux selon la classe choisie pour être la fabrique). On a donc une dépendance d'un paquetage stable vers un paquetage instable ce qui n'est pas souhaitable : mais pourquoi au fait ?
5. Pour supprimer la dépendance vers la fabrique **injectez** dans la chenille les instances d'anneaux au lieu que ce soit la chenille qui demande des instances. Si une seule instance devait être injectée (une chenille avec seulement une tête) la passer en paramètre du constructeur de la chenille suffirait. Mais la chenille possédant plusieurs anneaux, définissez une méthode publique **ajouterAnneau** et déplacez les appels à la fabrique dans une classe appropriée qui ensuite injectera ces anneaux via ajouterAnneau.
6. Vérifiez dans ajouterAnneau que seule une Tete est en tete de l'anneau et que la chenille reste avec une seule tete. Ceci doit se faire sans vérifier la classe de l'anneau à ajouter.
7. Mettez à jour le diagramme d'architecture.
8. (si le temps le permet) Définissez une classe anneaux.Bizarre dont les anneaux seront soit transparents soit plus gros. N'oubliez pas d'inclure quelques anneaux bizarres dans la chenille. Créez une seule classe mais un paramètre du constructeur décidera si l'anneau est gros ou transparent (une méthode polymorphe à ajouter à l'interface). La tête aussi sera systématiquement grosse. Profitez-en pour faire en sorte que la GUI détermine la couleur d'un anneau non plus selon sa position mais via des méthodes polymorphes sur les anneaux.
Vérifiez que le paquetage chenille n'a pas besoin d'être modifié pour ajouter cette classe. En revanche, il faut modifier la méthode qui injecte les instances d'anneaux dans la chenille.
9. (Si le temps le permet) Ajoutez des tests unitaires pour vérifier notamment qu'on respecte bien la règle selon laquelle seuls certains anneaux peuvent être en tête.